

Lab 3: Beginner Assembly with the CPU Visual Simulator

Module	ET4047
Date	27/09/2026
Lab No.	Lab 3
Author	Luke Griffin

This lab introduces assembly programming on the CPU Visual Simulator (cpuvisualsimulator.github.io). It follows the approach from the W03 lecture: first understand how the CPU and memory are organised, then write each program as **pseudo code**, translate it into **assembler**, and check it by comparing the state of the CPU **before and after execution**.

By the end of this lab you should be able to:

- describe what the PC, IR, Control Unit, MUX, ALU, ACC and Status Word do, and what travels on the address, data and control buses;
- explain how an instruction is stored in memory (opcode and operand) and why the PC goes up in steps of 2;
- tell the difference between **immediate** (#X) and **direct** (X) addressing and predict the result of each;
- read and write register-transfer notation such as [X] -> ACC and ACC + #1 -> ACC;
- translate IF-THEN-ELSE and WHILE-DO pseudo code into assembler using CMP and conditional jumps.

How this lab works. Part A explains how the simulator works. Read it first and keep it open as a reference. Part B contains six tasks. Each task gives the pseudo code, the assembler (a worked example), and a before/after table so that you can check your run. Load the example, run it, and confirm that you get the same result. Then complete the **Your Turn** exercises, which ask you to *predict*, *modify* and then *write* programs of your own. Write your answers in the boxes and tables provided.

Part A: How the CPU Visual Simulator Works

A1. The simulator at a glance

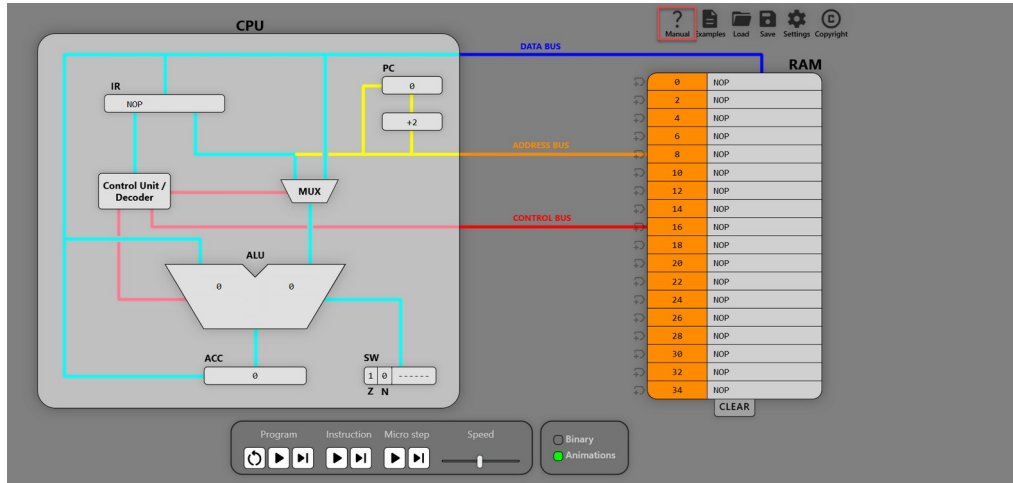


Figure 1: CPU Visual Simulator

The simulator models a simple **accumulator-based CPU** connected to RAM by three buses. Each part of Figure 1 is described below.

Part	What it does
PC (Program Counter)	Holds the address of the next instruction to fetch. The +2 box adds 2 after each fetch, because every instruction takes 2 bytes.
IR (Instruction Register)	Holds the instruction being executed. It is split into the opcode (what to do), which goes to the Control Unit, and the operand (an address or a number).
Control Unit / Decoder	Decodes the opcode and controls everything else (pink wires). It sets the MUX, chooses the ALU operation and sends read or write signals to RAM on the control bus.
MUX (multiplexer)	A switch that picks the ALU's second input. For immediate operands (#5) it takes the number from the IR. For direct operands (A) it takes the value read from RAM over the data bus.
ALU (Arithmetic Logic Unit)	Does the calculation. The left input is always the ACC and the right input comes from the MUX. The symbol shows the operation: +, -, *, : or = (pass the value straight through).
ACC (Accumulator)	The CPU's single working register (16 bits). Every load and every ALU result ends up here.
SW (Status Word)	Holds the flags. Z = 1 when the last ALU result was zero. N = 1 when it was negative. Conditional jumps test these flags.
RAM	Memory holding both the program and its data. The simulator shows 18 cells, at addresses 0 to 34.
Address bus (orange)	Tells RAM which address to use: the PC during a fetch, or the IR operand during execution.
Data bus (blue)	Carries the value : instructions into the IR, data to the MUX, and the ACC into RAM for STO.
Control bus (red)	Tells RAM what to do: read or write.

A2. Memory organisation

Every memory address holds **8 bits** (one byte), and addresses are also 8 bits, so the address range is **0 to 255** (00000000 to 11111111). An **instruction is 16 bits**: an 8-bit **opcode** and an 8-bit **operand**. Each instruction therefore occupies two addresses. That is why instructions sit at **even** addresses (0, 2, 4 ...) and why the PC adds 2 after every fetch.

Address	OPCODE	OPERAND	
0	opcode of instruction 1	operand of instruction 1	Instruction 1
2	opcode of instruction 2	operand of instruction 2	Instruction 2
4	opcode of instruction 3	operand of instruction 3	Instruction 3

A program has two parts. The **instruction block** starts at address 0 and ends with HLT. The **data block** holds the variables. Data values use the **I16** format: signed 16-bit integers from -32768 to 32767. When a code file is loaded, its lines go into memory **one after another, starting at address 0**. To put the data block at the bottom of RAM, as the example does in the WHILE-DO example, insert NOP lines between HLT and the data. The worked tasks in Part B do this, so their variables are at addresses 30 to 34, as in the figures. The NOPs are never executed, because the CPU stops at HLT.

A3. Instruction structure and addressing modes

Each instruction has three fields. The first bit of the opcode byte is the **immediate flag**, which says how to interpret the operand:

Field	Bits	Meaning
Immediate flag	1 bit (the leftmost)	0 = direct addressing, 1 = immediate addressing
Opcode	7 bits	Which operation to perform (LOD, ADD, JMP ...)
Operand	8 bits	Direct: an address (8 bits). Immediate: a number (I8, -128 to 127).

For example, LOD X is 00000111 and LOD #X is 10000111. The opcode is the same, and only the immediate flag differs. The two addressing modes are the most important idea in this lab:

Mode	Written as	The operand is ...	Example	Meaning
Direct	LOD A	the address of the value	LOD A	[A] -> ACC: go to address A and load what is stored there
Immediate	LOD #5	the value itself	LOD #5	#5 -> ACC: load the number 5
Immediate with a label	LOD #A	the address of A, used as a number	LOD #A	loads the number 34 (A's address), not A's contents

- An immediate value must fit in 8 bits: **-128 to 127**. To use a bigger number, such as 300, store it as data and load it directly.
- A direct operand must be a valid address: an **even** number from 0 to 254, or a label.
- ST0 and the jumps only use direct addressing. You can only store into, or jump to, an address.

A4. Instruction set in register-transfer notation

The lecture describes each instruction in **register-transfer notation (RTN)**. [X] means "the contents of memory location X", #X means "the number X", and -> means "is copied into". Use this notation in your code comments, as the example files do.

Control Flow		
NOP 0000000	No Operation, go to the next instruction	
HLT 0000001	Halt execution	
JMP X 0000010	Unconditional jump to the instruction at location X	
JZ X 0000011	Jump on Zero: If Flag Z is set, go to instruction number X, otherwise go to the next instruction	
JNZ X 0000100	Jump on Not Zero: If Flag Z is cleared, go to instruction number X, otherwise, go to the next instruction	
JN X 0000101	Jump on Negative: If Flag N is set, go to instruction number X, otherwise go to the next instruction	
JNN X 0000110	Jump on Not Negative: If Flag N is cleared, go to instruction number X, otherwise go to the next instruction	
Data Flow		
LDD X 0000111	Load content of Memory Location X (or number #X) into the Accumulator	
STO X 0001000	Store the value of the Accumulator into Memory Location X	
Arithmetic Logic		
ADD X 0001001	ADD #X 0001001	Add content of Memory Location X (or number #X) to the Accumulator. Flags are updated.
SUB X 0001010	SUB #X 0001010	Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated.
MUL X 0001011	MUL #X 0001011	Multiply the Accumulator by the content of Memory Location X (or number #X). Flags are updated.
DIV X 0001100	DIV #X 0001100	Divide the Accumulator by the content of Memory Location X (or number #X). Flags are updated.
AND X 0001101	AND #X 0001101	Bitwise AND between the Accumulator and the content of Memory Location X (or number #X). Flags are updated.
CMP X 0001110	CMP #X 0001110	Subtract content of Memory Location X (or number #X) from the Accumulator. Flags are updated, but the content of the Accumulator is not modified.
NOT X 0001111	NOT #X 0001111	Bitwise NOT of the content of Memory Location X (or number #X). The result is stored in the Accumulator.

Figure 2: Instruction set (simulator Manual)

Instruction	Opcode	Direct (X)	Immediate (#X)	Flags
NOP	0000000	no operation		
HLT	0000001	halt execution		
JMP X	0000010	X -> PC		
JZ X	0000011	if Z = 1 (ACC == 0): X -> PC		reads Z
JNZ X	0000100	if Z = 0 (ACC ≠ 0): X -> PC		reads Z
JN X	0000101	if N = 1 (ACC < 0): X -> PC		reads N
JNN X	0000110	if N = 0 (ACC ≥ 0): X -> PC		reads N
LDD	0000111	[X] -> ACC	#X -> ACC	
STO X	0001000	ACC -> [X]		
ADD	0001001	ACC + [X] -> ACC	ACC + #X -> ACC	updates Z, N
SUB	0001010	ACC - [X] -> ACC	ACC - #X -> ACC	updates Z, N
MUL	0001011	ACC * [X] -> ACC	ACC * #X -> ACC	updates Z, N
DIV	0001100	ACC / [X] -> ACC	ACC / #X -> ACC	updates Z, N
AND	0001101	ACC AND [X] -> ACC	ACC AND #X -> ACC	updates Z, N
CMP	0001110	ACC - [X] (ACC == X?)	ACC - #X (ACC == #X?)	updates Z, N only
NOT	0001111	NOT([X]) -> ACC	NOT(#X) -> ACC	

Important: jumps test the flags, not the ACC. The flags only change when an arithmetic or logic instruction runs (ADD, SUB, MUL, DIV, AND, CMP). LDD and STO leave them alone. After a reset the ACC is 0, so Z starts at 1. After loading a value,

always use CMP before a conditional jump. CMP works like SUB but only updates the flags, so the ACC keeps its value.

A5. How an instruction is executed

Every instruction goes through the same **fetch-decode-execute** cycle:

Stage	What happens	Bus used
Fetch	The PC's value goes on the address bus. RAM is told to read , sends the instruction back over the data bus into the IR, and the PC becomes PC + 2.	address, control, data
Decode	The Control Unit reads the opcode (and immediate flag) in the IR and sets up the MUX, the ALU and the bus signals.	internal (pink wires)
Execute	The instruction runs. It may read an operand from RAM, calculate in the ALU, write to RAM, or load a new address into the PC (jumps).	depends on the instruction

For example, here is how LOD A (direct) executes, as shown in the *LOD X* before/after example:

Step	What moves	Result
1	PC (0) → address bus. Control bus: read. RAM[0] → data bus → IR	IR = LOD A, PC = 2
2	Control Unit decodes LOD with immediate flag = 0 (direct)	MUX set to "memory"
3	IR operand (address of A) → address bus. Control bus: read	RAM puts [A] on the data bus
4	Data bus → MUX → ALU (operation =, pass through) → ACC	ACC = [A]

With LOD #5 (immediate), step 3 is skipped. The MUX takes the number 5 straight from the IR operand, so RAM is not read. For ST0 A, the ACC's value goes onto the data bus and the control bus signals **write**.

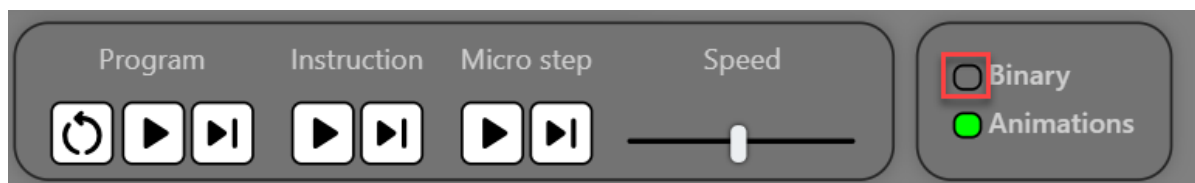


Figure 3: Control bar

The control bar has three ways to run a program. **Program** runs the whole program. **Instruction** runs one complete fetch-decode-execute cycle at a time. **Micro step** shows one bus transfer at a time, which is the best way to watch the steps in the tables above. Use **Speed** to slow the animation down. The **Binary** switch shows RAM and the registers in binary, so that you can see the opcode, the immediate flag and the operand.

A6. Writing programs: pseudo code, assembler and code files

One approach to writing a program is:

- **1. Write the pseudo code**, for example IF (X == 3) THEN Y = Y + 5 ELSE Z = Z + 2 ENDIF.

- **2. Translate it into assembler** using the IF-THEN-ELSE or WHILE-DO pattern below.
- **3. Comment every line** in register-transfer notation, for example ; [X] -> ACC.
- **4. Run it and compare the before and after states** (registers, flags and variables) with what you expected.

The IF-THEN-ELSE and WHILE-DO patterns

IF-THEN-ELSE	Assembler pattern	WHILE-DO	Assembler pattern
IF (condition)	LOD / CMP, then jump to ELSE if the condition is false	WHILE (condition)	WHILE: LOD / CMP, then jump to ENDWHILE if false
THEN instruction_1	THEN: ... followed by JMP ENDIF	DO instruction_2 ...	loop body, followed by JMP WHILE
ELSE instruction_2	ELSE: ...	ENDWHILE	ENDWHILE: next instruction
ENDIF instruction_3	ENDIF: ...		

Code files (.cpuvs)

- A .cpuvs file is a plain text file. Open it with **Load** and save your work with **Save**.
- Each line holds **one** instruction or one data value, and lines are stored from **address 0** onwards.
- The format is LABEL: INSTRUCTION. The label is optional; when you leave it out, leave out the : too.
- Everything after ; is a comment. Empty lines and comment-only lines are ignored.
- If the file has an error, the simulator shows a message along its top edge.

Figure 4 shows IF_THEN_ELSE_example.cpuvs, which you will use in Task 4.

```

LOD X      ; [X] -> ACC
CMP #3     ; ACC == 3?
JNZ ELSE   ; If not, jump to ELSE
THEN:      ; If yes, increase [Y] by 5
LOD #5
ADD Y
STO Y
JMP ENDIF  ; Jump to ENDIF
ELSE:      ; Increase [Z] by 2
LOD #2
ADD Z
STO Z
ENDIF:     ; 8 -> [X]
LOD #8
STO X
HLT        ; Halt execution

X:         3
Y:         0
Z:         0
  
```

Figure 4: Example code file (IF-THEN-ELSE example)

Part B: Lab Tasks

Each worked example below is written the same way. There is a ; comment in register-transfer notation on each line, the instruction block starts at address 0, and the data block is padded to the bottom of RAM with NOP so that the addresses match the figures. For your own programs you can leave out the NOP padding. The data then goes straight after HLT, as in the IF-THEN-ELSE example.

Task 1: Load and Store (Immediate and Direct Addressing)

Goal: Put a value into memory and read it back. This task shows the difference between **immediate** addressing (#5 is the number 5) and **direct** addressing (A is an address in memory).

Pseudo code

```
A = 5
ACC = A
```

Assembler

```

    LOD #5      ; #5 -> ACC      (immediate: the number 5)
    STO A       ; ACC -> [A]     (direct: store at address A)
    LOD A       ; [A] -> ACC     (direct: load from address A)
    HLT         ; Halt execution
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    A:          0                ; Data block: A is at address 34

```

LOD #5 uses **immediate** addressing: the operand byte holds the number 5 itself, so RAM is not read. STO A and LOD A use **direct** addressing: the operand byte holds the **address** of A (34), and the CPU goes to that address.

Before and after execution

	Before execution	After execution
PC	0	6 (address of HLT)
ACC	0	5
Z N	1 0	1 0 (unchanged: LOD and STO do not update the flags)
A (address 34)	0	5

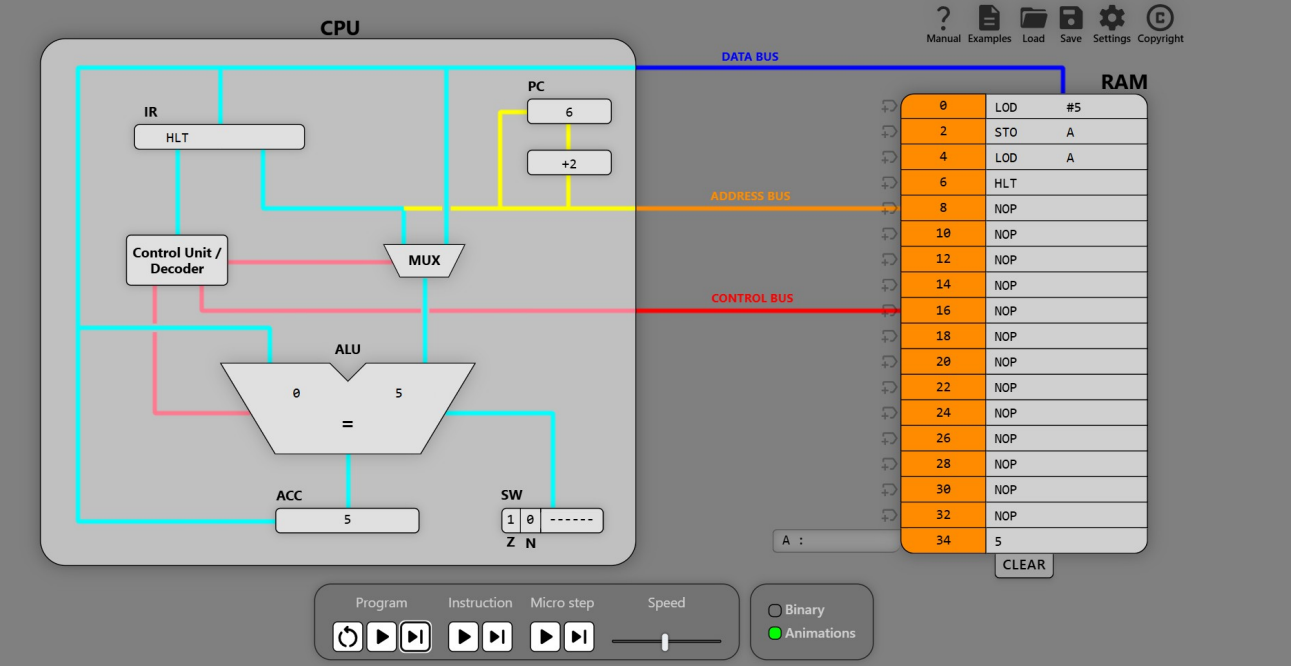


Figure 5: Task 1 after execution

Your Turn

1.1 When the program halts, the PC shows **6**. Explain why the PC counts 0, 2, 4, 6 and not 0, 1, 2, 3.

Your answer:

1.2 **Immediate or direct?** Replace line 3 (LOD A) with each instruction below, one at a time. *Predict* the ACC first, then run the program to check.

Line 3 becomes	Addressing mode	Predicted ACC	Actual ACC
LOD A			
LOD 34			
LOD #34			
LOD #A			

1.3 Turn on the **Binary** switch. Write down the two bytes stored at address 0 (LOD #5) and at address 4 (LOD A). Which bit is different, and why?

Address	Instruction	Opcode byte	Operand byte
0	LOD #5		
4	LOD A		

Your answer:

1.4 Change the program so that it also stores the value **9** in a second variable **B**. Take a screenshot showing $A = 5$ and $B = 9$ in RAM.

Your code:

1.5 Write a program that **swaps** two variables, starting with $A: 3$ and $B: 8$. When it halts, A should be 8 and B should be 3.

Hint: The ACC can only hold one value at a time, so you will need a third variable (TEMP) to keep one value safe.

Your code:

Task 2: Arithmetic and Expressions

Goal: Use the ALU to do arithmetic with values from memory (direct) and constants (immediate), and store the results back in RAM.

Part 1: adding two numbers from memory

SUM = A + B

```

    LOD A      ; [A] -> ACC
    ADD B      ; ACC + [B] -> ACC
    STO SUM    ; ACC -> [SUM]
    HLT        ; Halt execution
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    A: 3       ; address 30
    B: 4       ; address 32
    SUM: 0     ; address 34 (7 after execution)
```

	Before execution	After execution
PC	0	6
ACC	0	7
Z N	1 0	0 0 (ADD updated the flags: 7 is not zero and not negative)
A, B (addresses 30, 32)	3, 4	3, 4 (unchanged)
SUM (address 34)	0	7

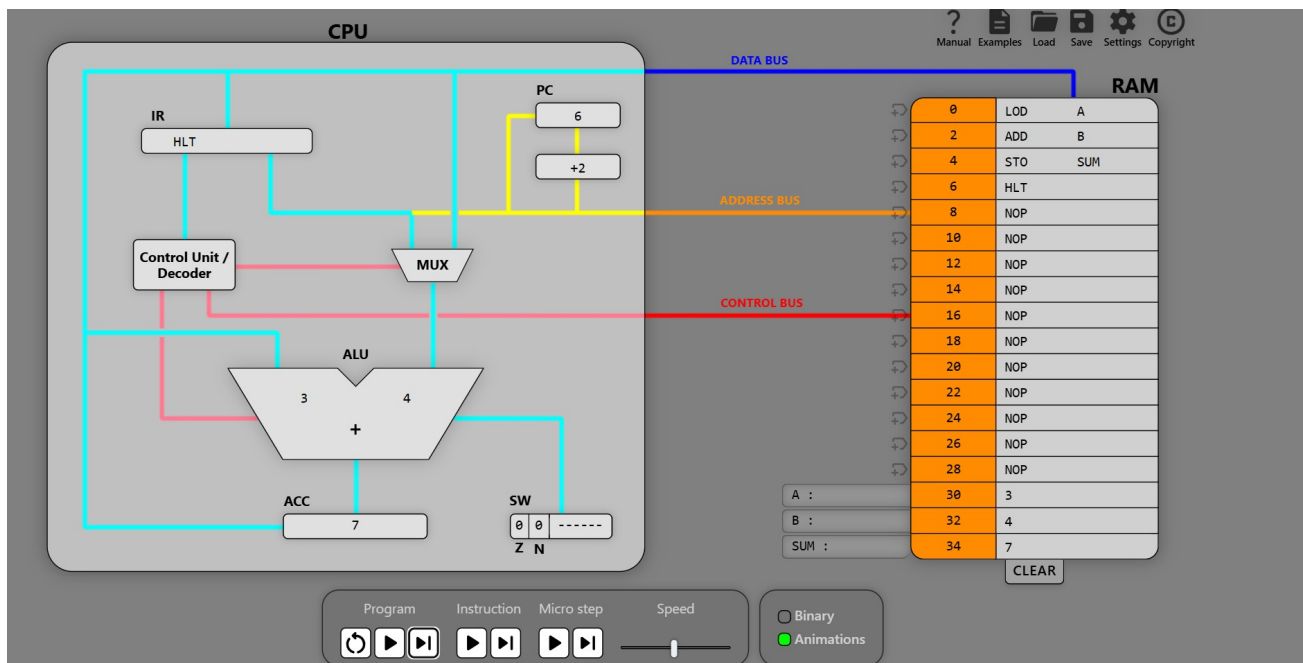


Figure 6: Task 2 after execution

Part 2: evaluating an expression (Example 1)

To calculate a longer expression, split it into parts. Work out each part in the ACC and store it in a variable, then combine the parts. Most operands here are **immediate** constants. MUL X is **direct**, because it uses the value stored in X.

```
X = 3 + 4*5
Y = 2*3 - 1
W = X * Y * 2
```

```
; Calculate expression: W = (3+4*5)*(2*3-1)*2
; X = 3+4*5
; Y = 2*3-1
; W = X*Y*2
    LOD #4      ; Find X = 3+4*5
    MUL #5
    ADD #3
    ST0 X
    LOD #2      ; Find Y = 2*3-1
    MUL #3
    SUB #1
    ST0 Y
    MUL X       ; Find X*Y
    MUL #2      ; Find X*Y*2
    ST0 W       ; Save result to W
    HLT
X:      0
Y:      0
W:      0
```

	Before execution	After execution
PC	0	22
X, Y, W (addresses 24, 26, 28)	0, 0, 0	23, 5, 230
ACC	0	230

Your Turn

2.1 In Part 1, change ADD B to SUB B. *Before* running it, predict SUM. Then run it. What do the **Z** and **N** flags show, and why?

Your answer:

2.2 Step through Example 1 one instruction at a time and fill in the ACC after each instruction.

Instruction	ACC after	Instruction	ACC after
LOD #4		MUL #3	
MUL #5		SUB #1	
ADD #3		ST0 Y	
ST0 X		MUL X	
LOD #2		MUL #2	

2.3 Write a program that calculates $W = (3 + 4*5) * (8/4 - 1) * 4$. Work out the answer by hand first.

Your code and hand calculation:

2.4 Write a program that stores the **average** of A: 4, B: 7 and C: 10 in AVG. Then change C to 11 and run it again. What result do you get, and why?

Your code and answer:

2.5 Why can you not write `LOD #300`? Give **two** different ways of getting the value 300 into the ACC.

Hint: Look at the size of an immediate operand in section A3, and compare it with the size of a data value.

Your answer:

Task 3: IF-THEN-ELSE (Check for Zero)

Goal: Make a decision. The program tests a value and sets FLAG depending on whether that value is zero. It uses CMP to set the flags and JNZ to choose which block runs.

Pseudo code

```
IF (X == 0)
THEN
    FLAG = 1
ELSE
    FLAG = 0
ENDIF
```

Assembler

```

        LOD X          ; [X] -> ACC
        CMP #0         ; ACC == 0? (sets Z if yes)
        JNZ NOTZERO    ; If not, jump to NOTZERO
ZERO:   LOD #1         ; THEN: #1 -> ACC
        STO FLAG       ; ACC -> [FLAG]
        JMP END        ; Jump to END
NOTZERO: LOD #0        ; ELSE: #0 -> ACC
        STO FLAG       ; ACC -> [FLAG]
END:    HLT            ; Halt execution
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
X:      0              ; address 32 (test value)
FLAG:   0              ; address 34
```

How it works: CMP #0 works out ACC – 0 without changing the ACC and updates the flags. If X is zero, Z is set to 1. JNZ NOTZERO jumps only when Z = 0 (X not zero), so it skips the THEN block. If X is zero, the jump is not taken. The CPU runs the THEN block from the label ZERO, and JMP END then jumps over the ELSE block. Compare this with the IF-THEN-ELSE pattern in section A6.

Before and after execution (X = 0)

	Before execution	After execution
PC	0	16 (address of HLT)
ACC	0	1
Z N	1 0	1 0 (set by CMP #0, because X = 0)
X (address 32)	0	0
FLAG (address 34)	0	1

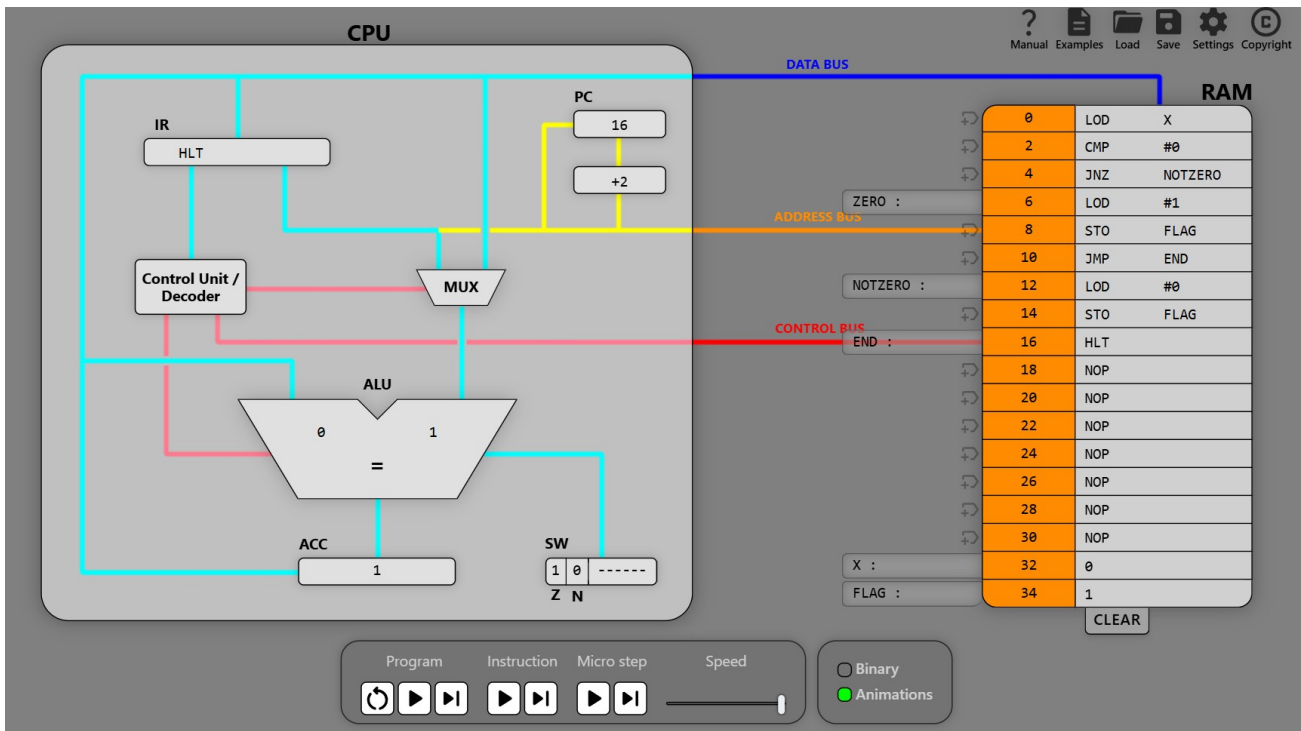


Figure 7: Task 3 after execution ($X = 0$)

Your Turn

3.1 Change X to **5** and step through the program one instruction at a time. What is in FLAG at the end? Which instruction addresses were **skipped**? Take a screenshot.

Your answer:

3.2 Rewrite the program using **JZ** instead of JNZ. It must give the same FLAG value for $X = 0$ and $X = 5$.

Your code:

3.3 Write a program for: IF ($X < 10$) THEN SMALL = 1 ELSE SMALL = 0. Test it with $X = 7$, $X = 10$ and $X = 12$.

Hint: CMP #10 works out $X - 10$. The result is negative ($N = 1$) exactly when $X < 10$. Use JN or JNN.

Your code and test results:

Task 4: IF-THEN-ELSE (Comparing Two Values)

Goal: Compare two values stored in memory. This time CMP B uses **direct** addressing, so the ACC is compared with the contents of B.

Pseudo code

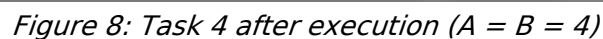
```
IF (A == B)
  THEN
    RESULT = 1
  ELSE
    RESULT = 2
ENDIF
```

Assembler

```
      LOD A      ; [A] -> ACC
      CMP B      ; ACC == [B]?
      JNZ NOTEQ  ; If not, jump to NOTEQ
      LOD #1      ; THEN: #1 -> ACC
      STO RESULT  ; ACC -> [RESULT]
      JMP END     ; Jump to END
NOTEQ: LOD #2      ; ELSE: #2 -> ACC
      STO RESULT  ; ACC -> [RESULT]
END:   HLT        ; Halt execution
      NOP
      NOP
      NOP
      NOP
      NOP
      NOP
      NOP
A:     4           ; address 30
B:     4           ; address 32 (try changing)
RESULT: 0         ; address 34
```

Before and after execution (A = B = 4)

	Before execution	After execution
PC	0	16
ACC	0	1
Z N	1 0	1 0 (CMP B: 4 - 4 = 0)
RESULT (address 34)	0	1



4.4 Write a program for: IF (A < B) THEN MAX = B ELSE MAX = A. Test it with A = 4, B = 9 and with A = 9, B = 4.

Hint: After CMP B, the ACC still holds A. Only the flags have changed.

Your code:

Task 5: Loops (Countdown and WHILE-DO)

Goal: Repeat instructions by jumping **backwards**. A loop needs a counter, a test and a backwards jump. Watch the PC. When the jump is taken, the PC is loaded with the address of the loop label.

Part 1: countdown (the test is at the bottom of the loop)

```
REPEAT
    N = N - 1
UNTIL (N == 0)
```

```

LOOP:  LOD N          ; [N] -> ACC
        SUB #1        ; ACC - #1 -> ACC
        STO N          ; ACC -> [N]
        CMP #0         ; ACC == 0?
        JNZ LOOP      ; If not, jump back to LOOP
        HLT            ; Halt execution
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
N:      6              ; address 34 (0 after execution)
```

	Before execution	After execution
PC	0	10
ACC	0	0
Z N	1 0	1 0
N (address 34)	6	0

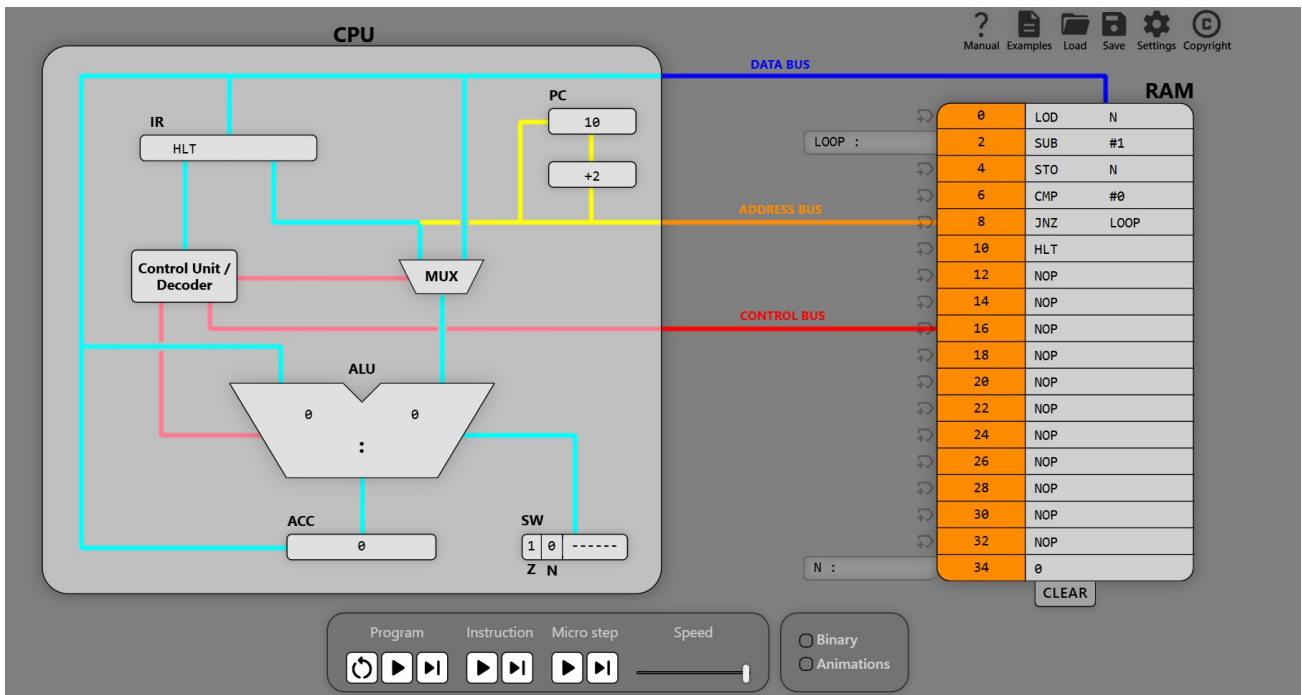


Figure 9: Task 5 after execution

Part 2: WHILE-DO (the test is at the top of the loop, from the WHILE_DO_example.cpuvs)

In a WHILE-DO loop the condition is tested **before** the body runs, so the body may run zero times. The three NOP lines push the data block down to addresses 30 to 34.

```
SUM = 0
COUNT = 0
WHILE (COUNT != MAX)
DO
    COUNT = COUNT + 1
    SUM = SUM + COUNT
ENDWHILE
```

```

        LOD #0      ; 0 -> ACC
        STO SUM     ; 0 -> [SUM]
        STO COUNT   ; 0 -> [COUNT]
WHILE:  LOD COUNT   ; [COUNT] -> ACC
        CMP MAX     ; ACC == [MAX]?
        JZ ENDWHILE ; If yes, jump to ENDWHILE
        ADD #1      ; ACC + 1 -> ACC
        STO COUNT   ; ACC -> [COUNT] (COUNT++)
        ADD SUM     ; ACC + [SUM] -> ACC
        STO SUM     ; ACC -> [SUM]
        JMP WHILE   ; Jump to WHILE
ENDWHILE: HLT      ; Halt execution
        NOP
        NOP
        NOP
MAX:    3
COUNT: 0
SUM:    0

```

	Before execution	After execution
PC	0	22 (ENDWHILE)
MAX, COUNT, SUM (addresses 30, 32, 34)	3, 0, 0	3, 3, 6

Your Turn

5.1 In Part 1, how many times does SUB #1 run when N = 6? When JNZ LOOP jumps, what value is loaded into the PC?

Your answer:

5.2 In Part 1, set N: 0 and run the program at a slow speed. What happens, and why? **Fix** the program by rewriting it as a WHILE-DO loop (test first), so that it halts straight away when N = 0 and still counts down when N = 6.

Your explanation and fixed code:

5.3 Step through Part 2 and fill in COUNT and SUM at the end of each pass through the loop. Then predict SUM for MAX = 5 and check it.

Pass	COUNT	SUM
1		
2		
3		
SUM for MAX = 5 (predicted / actual)		

5.4 Part 2 uses JZ ENDWHILE, but Part 1 uses JNZ LOOP. Explain why each loop needs a different jump.

Your answer:

Task 6: Bitwise Logic (NOT and AND)

Goal: Use the logic instructions. NOT flips all 16 bits of a value, and AND combines two values bit by bit. In I16 format, **0** is 0000000000000000 and **-1** is 1111111111111111 (two's complement), so 0 and -1 can be used as FALSE and TRUE.

Pseudo code

```
X = NOT A
Y = NOT B
Z = X AND Y
W = NOT Z
```

Assembler

```
      NOT A      ; NOT([A]) -> ACC
      ST0 X      ; ACC -> [X]
      NOT B      ; NOT([B]) -> ACC
      ST0 Y      ; ACC -> [Y]
      LOD X      ; [X] -> ACC
      AND Y      ; ACC AND [Y] -> ACC
      ST0 Z      ; ACC -> [Z]
      NOT Z      ; NOT([Z]) -> ACC
      ST0 W      ; ACC -> [W]
      HLT
A:    0
B:    0
X:    0
Y:    0
Z:    0
W:    0
```

NOT A uses direct addressing, so the CPU loads [A], inverts every bit and puts the result in the ACC. No separate LOD is needed. There is no NOP padding here, so the data block starts straight after HLT at address 20.

Before and after execution (A = 0, B = 0)

	Before execution	After execution
PC	0	18
X, Y, Z (addresses 24, 26, 28)	0, 0, 0	-1, -1, -1
W (address 30)	0	0

Your Turn

6.1 Run the program for every combination of A and B below and fill in W.

A	B	W	A	B	W
0	0		-1	0	
0	-1		-1	-1	

6.2 Which logic gate does this program implement? Use De Morgan's law to explain why. Predict W for $A = 5$ and $B = 3$, and check it (hint: write 5 and 3 in binary).

Your answer:

6.3 Write a program that calculates $W = \text{NOT}(A \text{ AND } B)$ (a NAND gate). Test it with $A = B = -1$.

Your code:

Challenges (Optional)

C.1 Division without DIV. Translate this pseudo code into assembler. Use JN to test $R < B$. Test it with $A = 17$ and $B = 5$; you should get $Q = 3$ and $R = 2$.

```
Q = 0
R = A
WHILE (R >= B)
DO
    R = R - B
    Q = Q + 1
ENDWHILE
```

Your code:

C.2 Write a program that calculates the **factorial** of N and stores it in RES ($N = 5$ gives 120). What goes wrong if N starts at 1, and how would you fix it?

Your code and answer:

Review Questions

R1 Explain the difference between `LOD 34`, `LOD #34` and `LOD A` (where A is at address 34).

Your answer:

R2 CMP and SUB both subtract. What is the difference, and why is CMP the right choice before a conditional jump?

Your answer:

R3 Which register changes when a jump is taken? Explain how this makes loops possible.

Your answer:

R4 Which instructions update the Z and N flags? Why do the programs in this lab use CMP straight after LOD?

Your answer:

R5 An immediate operand is limited to -128 to 127 , but a data value can be up to 32767 . Why? (Use section A3.)

Your answer: